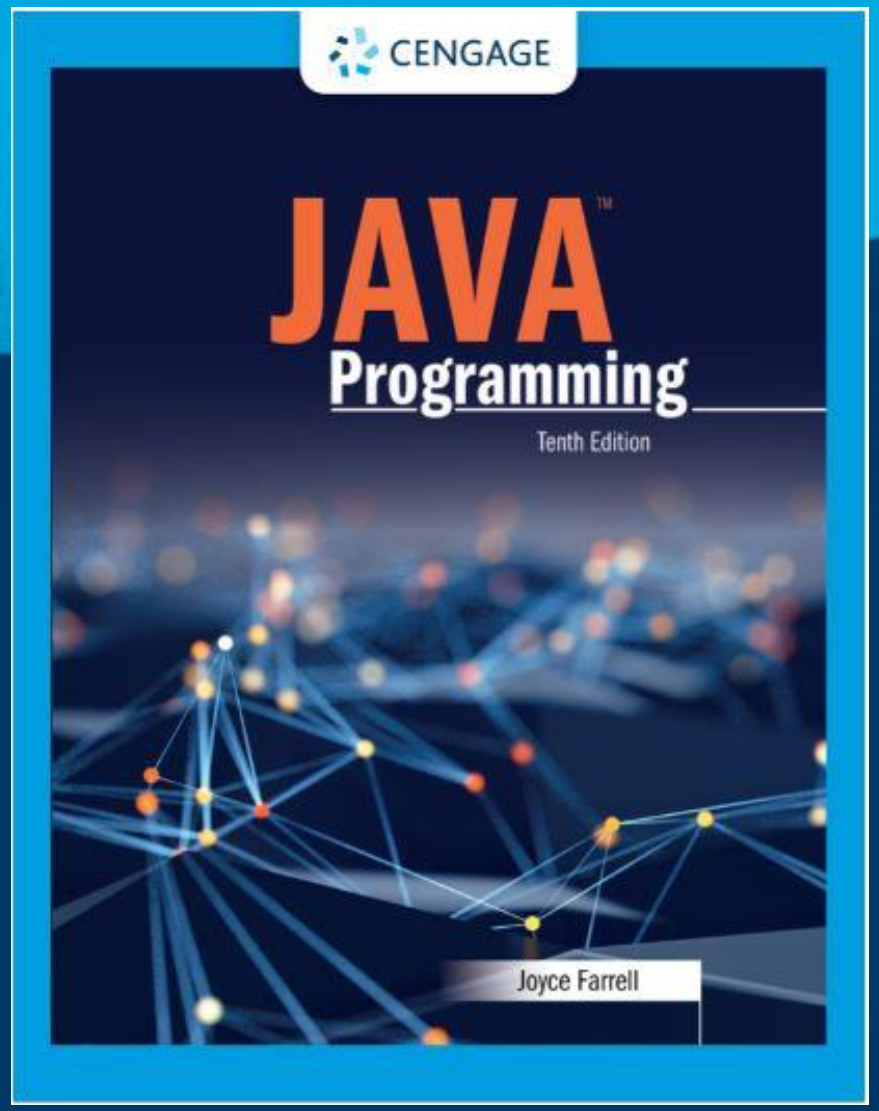


Java Programming, 10e

Chapter 09: Inheritance and Interfaces

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Chapter Objectives (1 of 2)

By the end of this chapter, you should be able to:

09.01 Describe the concept of inheritance

09.02 Extend classes

09.03 Override superclass methods

09.04 Call constructors during inheritance

09.05 Access superclass methods

09.06 Employ information hiding

09.07 Describe which methods you cannot override

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Chapter Objectives (2 of 2)

By the end of this chapter, you should be able to:

09.08 Create and use abstract classes

09.09 Use dynamic method binding

09.10 Create arrays of subclass objects

09.11 Use the `Object` class and its methods

09.12 Create and use interfaces

09.13 Use `records`, anonymous inner classes, and lambda expressions

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.1 Learning About the Concept of Inheritance (1 of 4)

Inheritance

- The classes you create in object-oriented programming languages can inherit data and methods from existing classes
- A mechanism that enables one class to acquire all of the behaviors and attributes of another class
- You can create a new class simply by indicating the ways in which it differs from an existing, tested class
- The new class automatically contains the data fields and methods from the original class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.1 Learning About the Concept of Inheritance (2 of 4)

Benefits of inheritance

- Saves time because fields and methods already exist
- Reduce errors because methods have been used and tested
- Reduce new learning needed for programmers to use the class if they are familiar with the original

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.1 Learning About the Concept of Inheritance (3 of 4)

Base class

- Class used as basis for inheritance
- Superclass
- Parent class

Derived class

- Class that inherits from a base class
- Subclass
- Child class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.1 Learning About the Concept of Inheritance (4 of 4)

Containment

- “has a” situation

Aggregation

- The relationship in which a class contains one or more members of another class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.2 Extending Classes (1 of 1)

`extends` keyword

- Used to achieve inheritance in Java
  - `public class EmployeeWithTerritory extends Employee`

In addition

- A child inherits from a parent
- A parent class object does not have access to its child's data and methods
- Subclasses are more specific than the superclass they extend
- `instanceof` yields `true` if the operand on the left can be upcast to the operand on the right

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.3 Overriding Superclass Methods (1 of 2)

Override

- To use the child's version of a field or method instead of parent's
- Each subclass method overrides any method in the parent class with the same name and parameter list
- If the parent class method has the same name but different parameters, the subclass overloads the parent class

Polymorphism

- Using the same method name to indicate different implementations

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.3 Overriding Superclass Methods (2 of 2)

Using the `@Override` annotation

- Lets the compiler know the intention to override a method in parent class
- Instead of creating a method with a new signature
- Compiler issues an error message if you do not use the `@Override` tag

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.4 Calling Constructors During Inheritance (1 of 3)

Calling a constructor

- When you create an object you are calling a constructor
- When you create a child object, both the parent and child constructors execute
- You usually want the superclass constructor to initialize the data fields that originate in the superclass.
- The subclass constructor usually needs to initialize only its specific data fields

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.4 Calling Constructors During Inheritance (2 of 3)

Using superclass constructors that require arguments

- Default constructor doesn't need arguments
- When you write your own constructor it replaces default
- If a superclass has only constructors that require arguments, subclasses must provide the superclass constructor with arguments
- When a superclass requires constructor arguments, you must write a subclass constructor so it can call on a superclass constructor
- The keyword super always refers to the superclass of the class in which you use it

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.4 Calling Constructors During Inheritance (3 of 3)

Conditions under which a subclass constructor is required

- **Not required:** If a superclass contains no constructors written by the programmer
- **Not required:** If a superclass contains a default constructor written by the programmer
- **Required:** If a superclass contains only nondefault constructors
  - Subclasses must contain a constructor that calls the superclass constructor

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.5 Accessing Superclass Methods (1 of 2)

Calling a superclass constructor

- When you call a superclass constructor from a subclass constructor the call must be the first statement in the constructor
- When you call an ordinary superclass method withing a subclass method, the call is not required to be the first statement

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.5 Accessing Superclass Methods (2 of 2)

Comparing `this` and `super`

- Sometimes refer to the same method, but not always
- If a subclass has not overridden a superclass method, the child can use the method name:
  - With `super`
  - With `this`
  - Alone

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.6 Employing Information Hiding (1 of 2)

Information hiding

- The concept of keeping data private
- Data only can be altered by methods and in ways you choose/control
- `private` members of a parent class are not accessible within a child class's methods

`Protected` access

- Between `public` and `private`
- Can only be used by a class and its descendants

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.6 Employing Information Hiding (2 of 2)

You only need to make parent class fields `protected` if:

- You want child classes to access parent data directly
- You want to prohibit other classes from accessing the fields
- Whenever possible use information hiding rather than `protected`

Fragile classes

- Classes that use field from parent classes
- Prone to errors

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.7 Methods You Cannot Override (1 of 5)

Three types of methods you cannot override in a subclass

- `static` methods
- `final` methods
- Methods within the `final` classes

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.7 Methods You Cannot Override (2 of 5)

`static` methods

- A subclass cannot override `static` methods in its superclass
- It can hide a `static` method in the superclass
  - Declaring a `static` method in the subclass
  - Same signature as `static` method in superclass
  - The `static` method that hides the superclass `static` method cannot access the parent method using the `super` object
- You cannot override a `static` method with a nonstatic method

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.7 Methods You Cannot Override (3 of 5)

`final` methods

- A subclass cannot override methods that are declared `final` in the superclass
- Use the `final` modifier with methods you don't want to be overridden

Virtual method calls

- All instance method calls are virtual by default
- Version of the method used is not determined when the program is compiled
- It is determined when the method call is made

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.7 Methods You Cannot Override (4 of 5)

Inlining

- Compiler optimizes a program's performance by removing calls to `final` methods
- Replaces them with the expanded code of their definitions at each method call location
- You are never aware when inlining takes place
- Compiler only inlines a `final` method if it contains only one or two lines of code

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.7 Methods You Cannot Override (5 of 5)

`final` superclass

- A subclass cannot override methods in a `final` superclass
- A `final` class cannot be a parent
  - You cannot extend a `final` class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.8 Creating and Using Abstract Classes (1 of 2)

Concrete class

- One from which you can instantiate objects

Abstract class

- A class you create only to extend from
- Keyword abstract
- Known in other languages as a virtual class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.8 Creating and Using Abstract Classes (2 of 2)

Types of methods within an abstract class

- Nonabstract: implemented in the abstract class and inherited by its children
- Abstract: has no body and must be implemented in its child classes

Abstract declarations

- If you declare a class to be abstract, its methods can be abstract or not
- If you declass a method to be abstract, you must declare its class abstract

Polymorphism

- Using the same method name to indicate different implementations

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.9 Using Dynamic Method Binding (1 of 1)

Dynamic method binding

- When a superclass is abstract, you can indirectly create a reference to a superclass abstract object
- An application's ability to select the correct subclass method depending on argument type
- Also called late method binding
- The opposite of static or fixed method binding
- Most useful when you want to create a method with one or more parameters that might be of several types

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.10 Creating Arrays of Subclass Objects (1 of 1)

Arrays of subclass objects

- Every array element must be the same data type
- An array is a space for references to objects
- When you create an array of superclass references it can hold subclass references
  - True whether the superclass is abstract or concrete

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.11 Using the `Object` Class and Its Methods (1 of 4)

`Object` class

- Defined in the `java.lang` package
- When you define a class, if you do not explicitly extend another class, the class is an extension of the `Object` class
- Includes methods that descendant classes can use, overload, or override

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.11 Using the `Object` Class and Its Methods (2 of 4)

Using the `toString()` method

- Converts an `Object` into a `String` that contains information about the `Object`
- A hash code is a calculated number used to uniquely identify an object
- If two objects have the same Java identifier, they will each have a unique hash code
- It is usually more useful to write your own overloaded `toString()` version than use the automatic method with your classes
- `toString()` compiles with any object whose details you want to see

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.11 Using the `Object` Class and Its Methods (3 of 4)

Using the `equals()` method

- Not static
- Takes a single argument that is compared to the calling object
- Returns a `Boolean` value indicating whether the objects are equal
- To create a method that compares two objects based on values they hold:
  - Create a method and use an identifier such as `areTheyEqual()`
  - Overload the `Object` class `equals()` method
  - Override the `Object` class `equals()` method

Overloading `equals()`

- A method overloads another when their parameter lists differ

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.11 Using the `Object` Class and Its Methods (4 of 4)

Overriding `equals()`

- When a subclass method overrides its parent, the signatures must be the same
- Determine if the `equals()` argument is the same object as the calling object by using a comparison such as `obj == this` and return `true` if it is
- Return `false` if the `Object` argument is `null`
- Return `false` if the calling and argument objects are not the same class
- Case the `Object` argument to the same type as the calling object only if they are the same class
- You should override the `hashCode()` method because equal objects should have equal hash codes

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 9.5: Case Study

You have been tasked with writing a program that uses inheritance.

- Can a subclass override methods that are declared `static` in the superclass? Why or why not?
- When you create an abstract method, what do you include?
- What is the operational difference between dynamic and static binding?
- Can you include array elements that are of different types? Explain.
- If two objects in your application have the same Java identifier, how are they differentiated?

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 9.5: Case Study Debrief

- A subclass cannot override a class method—a method you use without instantiating an object. The `static` method that hides the superclass `static` method cannot access the parent method using the `super` object.
- You provide the keyword `abstract` and the rest of the method header, (method type, name, and parameters). You do not provide curly braces or any statements—just a semicolon at the end of the declaration.
- Instance methods use dynamic binding; class methods use `static` method binding. Dynamic binding makes programs flexible; static binding operates more quickly.
- Every array element must be the same data type. When you create an array in Java, you are creating space for references to objects.
- If two objects in an application have the same Java identifier, each will have a unique hash code, which is a calculated number used for identification.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.12 Creating and Using Interfaces (1 of 4)

Multiple inheritance

- The capability to inherit from one or more class
- Inexperienced programmers may encounter problems

Interface

- An alternative to multiple inheritance
- Looks like a class, but all of its methods (if any) are `public` and abstract
- All of its data items (if any) are `public`, `static`, and `final`
- A description of what a class does, but not how it is done
- Declares method headers, but not instructions
- Keyword `implements` and the interface name in the class header

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.12 Creating and Using Interfaces (2 of 4)

Compare abstract classes and interfaces

- Similar in that you cannot instantiate concrete objects from either one
- Differ because abstract classes can contain nonabstract methods, but all methods in an interface must be abstract
- A class can inherit from only one abstract superclass, but can implement from any number of interfaces

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.12 Creating and Using Interfaces (3 of 4)

	Variables	Constructors	Methods
Interface	All variables in an interface must be `public`, `static`, and `final`.	An interface has no constructor and cannot be instantiated.	All methods in an interface must be `public`, abstract, and nonstatic.
Abstract class	There are no restrictions on variables in an abstract class.	An abstract class's constructor is invoked when a child class object is instantiated.	There are no restrictions on methods in an abstract class, although an abstract class frequently contains one or more abstract methods.

Table 9-3: Comparing components of abstract classes and interfaces

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.12 Creating and Using Interfaces (4 of 4)

Creating interfaces

- When you know what actions you want to include but want every user to separately define the behavior that occurs when the method executes
- When you want a class to implement behavior from more than one parent
- Data fields must be `public`, `static`, and `final`
  - Can't be `private` because they cannot contain method bodies
  - Must be `static` because you cannot create interface objects
  - Must be `final`, because there is no other way to set or change data fields' values

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.13 Using `records`, Anonymous Inner Classes, and Lamba Expressions (1 of 3)

`record` keyword

- Used to declare a class that is meant to hold data not meant to change
- Does not have elaborate get and set requirements
- When you create a `record`, you receive:
  - A `private final` field for each component
  - A public constructor whose parameters are derived from the components in the record definition
  - A `public` accessor method for each component with the same name and data type as the component
  - Implementations of `equals()`, `hashCode()`, and `toString()`

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.13 Using `records`, Anonymous Inner Classes, and Lamba Expressions (2 of 3)

Using anonymous inner classes

- A class that has no name and is defined inside another class
- To save work when you need a simple class that is instantiated only once in a program
- Use the new operator to create an inner class and simultaneously create an instance of it
- The instance either extends superclass or implements an interface
  - Do not use the keyword `extends` or `implements`
- Must either implement an interface or extend another class

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

9.13 Using `records`, Anonymous Inner Classes, and Lamba Expressions (3 of 3)

Final variable

- One whose value is assigned only once

Using lambda expressions

- Creates an object that implements a functional interface
- A functional interface contains just one abstract method
- Lamba operator is a minus sign and a greater-than sign

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 9.6: Discussion

1. What restrictions, if any, exist when using methods with interfaces and abstract classes?
2. Define the term, functional interface, and explain how it relates to a lambda expression.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Activity 9.6: Discussion Debrief

1. All methods in an interface must be `public`, abstract, and nonstatic. There are no restrictions on methods in an abstract class, although an abstract class frequently contains one or more abstract methods.
2. A functional interface contains just one abstract method. A lambda expression is used to create an object that implements a functional interface.

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Don’t Do It (1 of 2)

- Don’t capitalize the *o* in the `instanceof` operator
- Don’t try to directly access private superclass members from a subclass
- Don’t forget to call a superclass constructor within a subclass constructor if the superclass does not contain a default constructor
- Don’t try to override a `final` method in an extended class
- Don’t try to extend a `final` class
- Don’t write a body for an abstract method

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Don’t Do It (2 of 2)

- Don’t forget to end an abstract method header with a semicolon
- Don’t forget to override any abstract methods in any subclasses you derive
- Don’t mistakenly overload an abstract method instead of overriding it; the subclass method must have the same parameter list as the parent’s abstract method
- Don’t try to instantiate an abstract class object
- Don’t forget to override all the methods in an interface that you implement

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Self-Assessment (1 of 2)

1. For what purpose do programmers use inheritance?
2. Explain what the `extends` keyword is used for.
3. Differentiate between overloading and overriding.
4. Is a subclass constructor required when the superclass contains no constructors written by the programmer?
5. Compare the keywords `this` and `super`. When would you use each?
6. For what purpose do programmers use information hiding?
7. List three types of methods you cannot override in a subclass.

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Self-Assessment (2 of 2)

8. Differentiate between concrete and abstract classes.
9. In Java, what types of methods use dynamic binding?
10. Explain the problem with referring to “an array of objects.”
11. Define the term, hash code, as used in Java.
12. Explain why data fields in an interface must be `public`, `static`, and `final`.
13. For what purpose would you use an anonymous inner class?

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.

Summary

Click the link to review the objectives for this presentation.

Link to Objectives

CENGAGE

Joyce Farrell, Java Programming, 10th Edition. © 2023 Cengage. All Rights Reserved. May not be scanned, copied or duplicated, or posted to a publicly accessible website, in whole or in part.